

Introduction

CodiceFiscale is a .NET 9 library for working with Italian personal identifiers.

What it does

Feature	Class	Status
Validate a Codice Fiscale	<code>CodiceFiscaleValidator</code>	✓ Available
Parse a Codice Fiscale into personal data	<code>CodiceFiscaleParser</code>	✓ Available
Non-throwing parse (TryParse pattern)	<code>CodiceFiscaleParser.TryParse</code>	✓ Available
Generate a Codice Fiscale from personal data	<code>CodiceFiscaleGenerator</code>	✓ Available
Match a Codice Fiscale against personal data	<code>CodiceFiscaleMatcher</code>	✓ Available
Validate an IBAN (SEPA countries)	<code>IBANValidator</code>	✓ Available
Validate an Italian VAT code (Partita IVA)	<code>ItalianVatCodeValidator</code>	✓ Available

Design principles

- **Local-only** — all logic runs in-process. No HTTP calls, no external services. Personal data never leaves your application.
- **Embedded datasets** — the municipality/Belfiore dataset (~7 800 comuni) and the foreign country codes (261 countries including historical ones) are compiled directly into the DLL as embedded resources. No files to deploy, no database to configure.
- **Lazy loading** — datasets are deserialized once on first use and cached for the lifetime of the process. Repeated calls are fast.
- **Immutable results** — all output types are C# `records`.
- **Never throws on invalid input** — `IsValid` methods always return `bool`. Only `Parse` throws; use `TryParse` for exception-free parsing.

Supported platforms

- .NET 9.0+

License

MIT — see the [repository](#) for details.

Getting Started

Installation

```
dotnet add package CodiceFiscale
```

Or via the NuGet Package Manager in Visual Studio: search for **CodiceFiscale**.

Validate a Codice Fiscale

```
using CodiceFiscale;

bool valid = CodiceFiscaleValidator.IsValid("RSSMRA85T10A562S");
// true

// Case-insensitive
bool valid2 = CodiceFiscaleValidator.IsValid("rSSmra85t10a562s");
// true

// Foreign birth place (Z-code)
bool validForeign = CodiceFiscaleValidator.IsValid("MRNGRL01P55Z614X");
// true

// Invalid check digit
bool invalid = CodiceFiscaleValidator.IsValid("RSSMRA85T10A562X");
// false
```

`IsValid` returns `false` (never throws) for any invalid input including `null`-equivalent strings, wrong length, bad characters, invalid date, unknown Belfiore code, or wrong check digit.

Parse a Codice Fiscale

```
using CodiceFiscale;
using CodiceFiscale.Entities;
using CodiceFiscale.Enums;

CodiceFiscaleData data = CodiceFiscaleParser.Parse("RSSMRA85T10A562S");

Gender gender      = data.Gender;           // Gender.Male
DateOnly dob       = data.DateOfBirth;      // new DateOnly(1985, 12, 10)
string belfiore    = data.BelfioreCode;     // "A562"
```

⚠ WARNING

Parse throws `InvalidCodiceFiscaleException` if the input is not valid. Use `TryParse` to avoid try/catch:

```
if (CodiceFiscaleParser.TryParse(userInput, out CodiceFiscaleData? data))
{
    Console.WriteLine($"Born: {data.DateOfBirth}, Gender: {data.Gender}");
}
```

Generate a Codice Fiscale

```
using CodiceFiscale;
using CodiceFiscale.Enums;

string cf = CodiceFiscaleGenerator.Generate(
    name:         "Luigi",
    surname:       "Verdi",
    dateOfBirth:   new DateOnly(1970, 5, 30),
    gender:        Gender.Male,
    belfioreCode:  "F839"
);
// "VRDLGU70E30F839C"
```

`Generate` throws `InvalidCodiceFiscaleDataException` if name/surname are shorter than 3 characters or the Belfiore code is not found in the embedded dataset. The generated CF always passes `CodiceFiscaleValidator.IsValid`.

Validate an IBAN

```
using CodiceFiscale;

bool valid = IBANValidator.IsValid("IT60X0542811101000000123456"); // true
bool validSpaced = IBANValidator.IsValid("IT60 X054 2811 1010 0000 0123 456"); // true
(spaces ignored)

bool invalid = IBANValidator.IsValid("IT00X0542811101000000123456"); // false (bad checksum)
```

Validate a Partita IVA

```
using CodiceFiscale;

// Standard company VAT
bool valid = ItalianVatCodeValidator.IsValid("00484960588", isConsumer: false, isFiscal:
false); // true

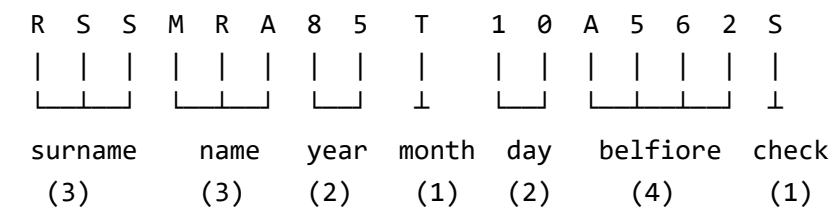
// Consumer (natural person) VAT – first digit must be 8 or 9
bool consumer = ItalianVatCodeValidator.IsValid("85423511618", isConsumer: true, isFiscal:
false); // true

// Accept a Codice Fiscale as a valid fiscal identifier
bool fiscal = ItalianVatCodeValidator.IsValid("RSSMRA85T10A562S", isConsumer: false,
isFiscal: true); // true
```

Codice Fiscale Structure

The Italian *Codice Fiscale* is a 16-character alphanumeric string that uniquely identifies a natural person for tax purposes.

Layout



Position	Length	Content	Example
1–3	3 chars	Surname consonants/vowels	RSS
4–6	3 chars	Name consonants/vowels	MRA
7–8	2 digits	Year of birth (last 2 digits)	85 → 1985
9	1 char	Month of birth (letter code)	T → December
10–11	2 digits	Day of birth (female: day + 40)	10 → day 10
12–15	4 chars	Belfiore catastal code	A562
16	1 char	Check character	S

Surname encoding (positions 1–3)

Consonants are taken first (in order), then vowels. If fewer than 3 characters are available, **x** is used as padding.

Surname	Consonants	Vowels	Code
Rossi	R, S, S	O, I	RSS
Re	R	E	REX
Fo	F	O	FOX

Name encoding (positions 4–6)

If the name has **4 or more consonants**, the 1st, 3rd, and 4th are taken. Otherwise, the same consonants-then-vowels-then-X rule applies as for the surname.

Name	Consonants	Code
Mario	M, R	MRA (M + R + vowel A)
Luca	L, C	LCU (L + C + vowel U)
Alessandro	L, S, S, N, D, R	LSN (1st, 3rd, 4th consonant)

Month encoding (position 9)

Letter	Month
A	January
B	February
C	March
D	April
E	May
H	June
L	July
M	August
P	September
R	October
S	November
T	December

Day encoding (positions 10–11)

- **Male:** actual day of birth (01–31)
- **Female:** day of birth + 40 (41–71)

This allows unambiguous gender determination from the two-digit day field.

Belfiore code (positions 12–15)

A 4-character code identifying the place of birth:

- **Italian municipality:** one letter (A–Z, excluding certain letters) + 3 digits, e.g. A562 (Rome)
- **Foreign country:** always starts with Z + 3 digits, e.g. Z614 (Venezuela)

The full dataset (~7 800 Italian municipalities + ~260 foreign countries) is embedded in the library.

Check character (position 16)

Computed from the first 15 characters using a weighted sum algorithm:

1. Characters at **odd positions** (1, 3, 5, ...) use a non-linear lookup table
2. Characters at **even positions** (2, 4, 6, ...) use their ordinal value (0–25 for letters, 0–9 for digits)
3. Sum all values, take modulo 26, convert to letter (A=0, B=1, ...)

This library validates the check character automatically inside `CodiceFiscaleValidator.IsValid`.

Omocodia

When two people would share the same Codice Fiscale (same name, birthdate, and birthplace), one or more of the numeric digits (positions 7–8, 10–11) are replaced with letters according to a fixed substitution table:

Digit	Letter
0	L
1	M
2	N
3	P
4	Q

Digit	Letter
5	R
6	S
7	T
8	U
9	V

 NOTE

Omocodia support is not yet implemented in this library. Standard Codici Fiscali (all digits in numeric positions) are fully supported.

Validation

Overview

`CodiceFiscaleValidator.IsValid(string cf)` performs a multi-step validation of an Italian Codice Fiscale. It returns `true` only if **all** checks pass.

```
using CodiceFiscale;

bool valid = CodiceFiscaleValidator.IsValid("RSSMRA85T10A562S"); // true
```

The method is **static** and **thread-safe**. It never throws — invalid or `null`-like inputs always return `false`.

Validation pipeline

The validator applies the following checks in order, short-circuiting on the first failure:

Step 1 — Basic format

- Input must not be null or empty
- Length must be exactly **16 characters**
- Must contain **no whitespace** (tabs, spaces, newlines all fail)

```
CodiceFiscaleValidator.IsValid("RSSMRA85T10A562S "); // false (trailing space)
CodiceFiscaleValidator.IsValid("RSSMRA85T10A562"); // false (15 chars)
```

Step 2 — Regex structure

The input is matched against the pattern:

<code>[A-Z]{3}</code>	surname
<code>[A-Z]{3}</code>	name
<code>[0-9]{2}</code>	year
<code>[ABCDEHILMPRST]</code>	month letter
<code>[0-9]{2}</code>	day
<code>\S{4}</code>	belfiore code
<code>[A-Z]</code>	<code>check</code> character

Input is uppercased before matching, so lowercase letters are accepted.

```
CodiceFiscaleValidator.IsValid("rssmra85t10a562s"); // true
CodiceFiscaleValidator.IsValid("RSS1RA85T10A562S"); // false (digit in name)
```

```
CodiceFiscaleValidator.IsValid("RSSMRA85110A562S"); // false (digit in month)
```

Step 3 — Date coherence

The day and month are validated against the actual calendar:

- Months with 30 days (April **D**, June **H**, September **P**, November **S**): day 1–30 (male) or 41–70 (female)
- February **B**: day 1–28 or 1–29 for leap years (male) or 41–68/41–69 (female)
- Other months: day 1–31 (male) or 41–71 (female)

```
CodiceFiscaleValidator.IsValid("RSSMRA85D31A562?"); // false (April has 30 days)
CodiceFiscaleValidator.IsValid("RSSMRA85B29A562?"); // false (1985 is not a leap year)
```

Step 4 — Belfiore code

The 4-character Belfiore code is looked up in the embedded datasets:

- Codes starting with a **letter other than Z**: searched in the Italian municipalities dataset (~7 800 comuni)
- Codes starting with **Z**: searched in the foreign countries dataset (~260 countries)

```
CodiceFiscaleValidator.IsValid("RSSMRA85T10Z999S"); // false (Z999 not assigned)
CodiceFiscaleValidator.IsValid("MRNGRL01P55Z614X"); // true (Z614 = Venezuela)
```

Step 5 — Check character

The 16th character is recomputed from the first 15 using the standard algorithm and compared to the actual value. See [Codice Fiscale Structure](#) for details.

```
CodiceFiscaleValidator.IsValid("RSSMRA85T10A562S"); // true (S is correct)
CodiceFiscaleValidator.IsValid("RSSMRA85T10A562X"); // false (X is wrong)
```

Return value summary

Condition	Returns
All checks pass	true
Null, empty, or whitespace-only	false
Wrong length	false

Condition	Returns
Contains whitespace	false
Fails regex pattern	false
Impossible calendar date	false
Unknown Belfiore/Z code	false
Wrong check character	false

Parsing

Overview

`CodiceFiscaleParser` provides two methods to decode a valid Codice Fiscale into its constituent personal data:

Method	Throws on invalid?	Returns
<code>Parse(string cf)</code>	 Yes — <code>InvalidCodiceFiscaleException</code>	<code>CodiceFiscaleData</code>
<code>TryParse(string cf, out CodiceFiscaleData? data)</code>	 No	<code>bool</code>

Parse

```
using CodiceFiscale;
using CodiceFiscale.Entities;
using CodiceFiscale.Enums;

CodiceFiscaleData data = CodiceFiscaleParser.Parse("RSSMRA85T10A562S");
```

TryParse

Use `TryParse` when you want to avoid try/catch blocks — for example, in form validation or batch processing:

```
if (CodiceFiscaleParser.TryParse("RSSMRA85T10A562S", out CodiceFiscaleData? data))
{
    Console.WriteLine($"Gender:      {data.Gender}");           // Male
    Console.WriteLine($"DateOfBirth: {data.DateOfBirth}");    // 1985-12-10
    Console.WriteLine($"Belfiore:    {data.BelfioreCode}");    // A562
}
else
{
    Console.WriteLine("Invalid CF");
}
```

Return type: `CodiceFiscaleData`

```
public record CodiceFiscaleData(  
    Gender    Gender,  
    DateOnly  DateOfBirth,  
    string    BelfioreCode);
```

Property	Type	Description	Example
<code>Gender</code>	<code>Gender</code>	Male or Female	<code>Gender.Male</code>
<code>DateOfBirth</code>	<code>DateOnly</code>	Full date of birth	<code>new DateOnly(1985, 12, 10)</code>
<code>BelfioreCode</code>	<code>string</code>	4-char Belfiore code of place of birth	<code>"A562"</code>

Gender

Gender is inferred from the day field:

- Day **1–31** → `Gender.Male`
- Day **41–71** → `Gender.Female` (actual day = encoded day – 40)

Date of birth

The year is reconstructed from the 2-digit year using the current year as cutoff:

- If the 2-digit year $\leq$ current year's last 2 digits → 2000s (e.g. `01` → 2001)
- Otherwise → 1900s (e.g. `85` → 1985)

Belfiore code

The raw 4-character code is returned as-is. Italian municipality codes start with a letter (other than Z); foreign country codes start with `Z`.

Exception: `InvalidCodiceFiscaleException`

`Parse` runs `CodiceFiscaleValidator.IsValid` internally. If the input is not valid, it throws:

```
throw new InvalidCodiceFiscaleException($"The provided Codice Fiscale '{cf}' is not valid");
```

Always validate before parsing, use `TryParse`, or handle the exception:

```

// Option 1 – TryParse (recommended)
if (CodiceFiscaleParser.TryParse(input, out var data))
{
    // use data
}

// Option 2 – validate first
if (CodiceFiscaleValidator.IsValid(input))
{
    CodiceFiscaleData data = CodiceFiscaleParser.Parse(input);
}

// Option 3 – try/catch
try
{
    CodiceFiscaleData data = CodiceFiscaleParser.Parse(input);
}
catch (InvalidCodiceFiscaleException ex)
{
    // handle invalid input
}

```

Examples

```

// Male, born in Italy
CodiceFiscaleData mario = CodiceFiscaleParser.Parse("RSSMRA85T10A562S");
// Gender:      Male
// DateOfBirth:  1985-12-10
// BelfioreCode: A562 (Rome)

// Female, born abroad (Venezuela)
CodiceFiscaleData gabriella = CodiceFiscaleParser.Parse("MRNGRL01P55Z614K");
// Gender:      Female (55 - 40 = day 15)
// DateOfBirth:  2001-09-15
// BelfioreCode: Z614 (Venezuela)

// TryParse on invalid input
bool ok = CodiceFiscaleParser.TryParse("INVALID", out CodiceFiscaleData? result);
// ok:      false
// result: null

```

Generation

Overview

`CodiceFiscaleGenerator.Generate` builds a valid 16-character Codice Fiscale from personal data.

```
using CodiceFiscale;
using CodiceFiscale.Enums;

string cf = CodiceFiscaleGenerator.Generate(
    name:      "Luigi",
    surname:    "Verdi",
    dateOfBirth: new DateOnly(1970, 5, 30),
    gender:     Gender.Male,
    belfioreCode: "F839"
);
// "VRDLGU70E30F839C"
```

The method throws `InvalidCodiceFiscaleDataException` if any input is invalid. It never returns a structurally wrong CF — the check character is always recomputed from the first 15 characters.

Signature

```
public static string Generate(
    string name,
    string surname,
    DateOnly dateOfBirth,
    Gender gender,
    string belfioreCode
)
```

Parameter	Type	Description
<code>name</code>	<code>string</code>	Person's first name (≥ 3 characters)
<code>surname</code>	<code>string</code>	Person's surname (≥ 3 characters)
<code>dateOfBirth</code>	<code>DateOnly</code>	Date of birth
<code>gender</code>	<code>Gender</code>	<code>Gender.Male</code> or <code>Gender.Female</code>
<code>belfioreCode</code>	<code>string</code>	4-character Belfiore (cadastral) code of place of birth

Encoding algorithm

The generator applies the official Italian Codice Fiscale algorithm in 7 steps:

1 — Surname code (positions 1–3)

Take the consonants of the surname first, then the vowels. Pad with **x** if fewer than 3 characters.

Surname	Consonants	Vowels	Code
Rossi	RSS	OI	RSS
Fo	F	O	FOX
Li	L	I	LIX

2 — Name code (positions 4–6)

- If the name has **4 or more consonants**: take the **1st, 3rd, and 4th** consonant.
- Otherwise: same rule as surname (consonants → vowels → X padding).

Name	Consonants	Rule	Code
Luigi	LG	< 4 consonants → consonants + vowels	LGU
Giovanni	GVN + N = GVNN	4 consonants → 1st/3rd/4th	GNN
Maria	MR	< 4 → MR + A	MRA

3 — Year (positions 7–8)

Last two digits of the birth year.

1985 → 85

4 — Month (position 9)

Each month is mapped to a fixed letter:

Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
A	B	C	D	E	H	L	M	P	R	S	T

5 — Day (positions 10–11)

- **Male:** day as-is, zero-padded to 2 digits (05, 30).
- **Female:** day + 40 (15 → 55, 01 → 41).

6 — Belfiore code (positions 12–15)

The 4-character municipality or foreign country code, uppercased. Italian codes start with a consonant; foreign country codes (from the Z-list) start with Z.

To find your municipality's Belfiore code you can use the official [Agenzia delle Entrate](#) lookup, or any Italian municipality database.

7 — Check character (position 16)

Computed from the first 15 characters using the official lookup table. Odd-position characters and even-position characters have separate weight tables; the sum modulo 26 maps to a letter A–Z.

Exceptions

`InvalidCodiceFiscaleDataException` is thrown when:

Condition	Message
<code>name</code> is null, empty, or shorter than 3 characters	The provided Name '...' is not valid
<code>surname</code> is null, empty, or shorter than 3 characters	The provided Surname '...' is not valid
<code>belfioreCode</code> is not 4 characters or not found in the embedded dataset	The provided Belfiore code '...' is not valid

Examples

```
using CodiceFiscale;
using CodiceFiscale.Enums;

// Male – Italian municipality (Roma = F839)
string cf1 = CodiceFiscaleGenerator.Generate("Luigi", "Verdi", new DateOnly(1970, 5, 30),
Gender.Male, "F839");
// "VRDLGU70E30F839C"

// Female – Italian municipality (Napoli = F839... use real code)
string cf2 = CodiceFiscaleGenerator.Generate("Maria", "Rossi", new DateOnly(1985, 3, 15),
Gender.Female, "H501");
```

```
// "RSSMRA85C55H501V"  
  
// Round-trip: generate then parse  
bool ok = CodiceFiscaleParser.TryParse(cf1, out CodiceFiscaleData? data);  
// ok: true  
// data.Gender: Gender.Male  
// data.DateOfBirth: new DateOnly(1970, 5, 30)  
// data.BelfioreCode: "F839"
```

NOTE

The Belfiore code must exist in the embedded dataset (Italian municipalities or foreign countries).
Passing an invented or retired code will throw `InvalidCodiceFiscaleDataException`.

Matching

Overview

`CodiceFiscaleMatcher.Matches` verifies whether a given Codice Fiscale corresponds to a specific person's data. It generates the expected CF from the provided inputs and compares it to the supplied string — returning `true` only if they are identical.

```
using CodiceFiscale;
using CodiceFiscale.Enums;

bool match = CodiceFiscaleMatcher.Matches(
    cf:          "VRDLGU70E30F839C",
    name:        "Luigi",
    surname:     "Verdi",
    dateOfBirth: new DateOnly(1970, 5, 30),
    gender:      Gender.Male,
    belfioreCode: "F839"
);
// true
```

The method is **case-insensitive** and trims surrounding whitespace from the CF before comparing. It **never throws** — invalid or null inputs return `false`.

Signature

```
public static bool Matches(
    string cf,
    string name,
    string surname,
    DateOnly dateOfBirth,
    Gender gender,
    string belfioreCode
)
```

Parameter	Type	Description
<code>cf</code>	<code>string</code>	The Codice Fiscale to verify
<code>name</code>	<code>string</code>	Person's first name (≥ 3 characters)
<code>surname</code>	<code>string</code>	Person's surname (≥ 3 characters)

Parameter	Type	Description
<code>dateOfBirth</code>	<code>DateOnly</code>	Date of birth
<code>gender</code>	<code>Gender</code>	<code>Gender.Male</code> or <code>Gender.Female</code>
<code>belfioreCode</code>	<code>string</code>	4-character Belfiore code of place of birth

How it works

1. **Validate inputs** — if `cf` is null or empty, or if `name/surname/belfioreCode` would cause `CodiceFiscaleGenerator` to throw, `Matches` returns `false` immediately.
2. **Generate** the expected CF with `CodiceFiscaleGenerator.Generate(...)`.
3. **Normalize** the supplied `cf` with `CodiceFiscaleNormalizer.NormalizeOmocodia` to handle omocodia variants.
4. **Compare** the two strings (case-insensitive).

Return value

Condition	Returns
CF matches the generated code	<code>true</code>
<code>cf</code> is null, empty, or whitespace	<code>false</code>
<code>name</code> or <code>surname</code> shorter than 3 characters	<code>false</code>
<code>belfioreCode</code> not found in the dataset	<code>false</code>
Any personal data field does not match the CF	<code>false</code>

Examples

```
using CodiceFiscale;
using CodiceFiscale.Enums;

// ✅ Correct data
CodiceFiscaleMatcher.Matches("VRDLGU70E30F839C", "Luigi", "Verdi",
    new DateOnly(1970, 5, 30), Gender.Male, "F839");
// true
```

```
//  Case-insensitive
CodiceFiscaleMatcher.Matches("vrdlgu70e30f839c", "Luigi", "Verdi",
    new DateOnly(1970, 5, 30), Gender.Male, "F839");
// true

//  Wrong date of birth
CodiceFiscaleMatcher.Matches("VRDLGU70E30F839C", "Luigi", "Verdi",
    new DateOnly(1971, 5, 30), Gender.Male, "F839");
// false

//  Wrong gender
CodiceFiscaleMatcher.Matches("VRDLGU70E30F839C", "Luigi", "Verdi",
    new DateOnly(1970, 5, 30), Gender.Female, "F839");
// false

//  Null CF
CodiceFiscaleMatcher.Matches(null!, "Luigi", "Verdi",
    new DateOnly(1970, 5, 30), Gender.Male, "F839");
// false
```

NOTE

`Matches` generates the CF internally using `CodiceFiscaleGenerator`. The same encoding rules apply (surname consonants, name 1st/3rd/4th consonant rule, month letter, etc.). If the person's name produces a different encoding than the one encoded in the CF (e.g. due to a hyphenated name or accent), `Matches` will return `false`.

IBAN Validation

Overview

`IBANValidator.IsValid(string iban)` validates an International Bank Account Number according to the ISO 13616 standard using the mod-97 algorithm.

NOTE

Only **SEPA** country codes are accepted (36 countries: EU27 + UK, Switzerland, Norway, Iceland, Liechtenstein, Monaco, San Marino, Andorra, Vatican City). Non-SEPA countries such as the USA, Canada, or Australia do not use IBAN and will return `false`.

```
using CodiceFiscale;
```

```
bool valid = IBANValidator.IsValid("IT60X0542811101000000123456"); // true
```

The method is **static** and **thread-safe**. It never throws.

Supported formats

Spaces are accepted and stripped before validation, so both compact and formatted IBANs are valid inputs:

```
IBANValidator.IsValid("IT60X0542811101000000123456");           // true – compact  
IBANValidator.IsValid("IT60 X054 2811 1010 0000 0123 456");      // true – formatted
```

Validation algorithm

1. **Strip whitespace** and uppercase the input
2. **Check country code** — the first 2 characters must be a SEPA member country code
3. **Check expected length** — each country has a fixed IBAN length (e.g. Italy: 27, Germany: 22, UK: 22, France: 27)
4. **Rearrange** — move the first 4 characters to the end
5. **Convert letters to digits** — A=10, B=11, ..., Z=35
6. **Compute mod 97** — the result must equal `1`

SEPA countries supported

Country	Code	Length	Country	Code	Length
Austria	AT	20	Netherlands	NL	18
Belgium	BE	16	Norway	NO	15
Bulgaria	BG	22	Poland	PL	28
Croatia	HR	21	Portugal	PT	25
Cyprus	CY	28	Romania	RO	24
Czech Republic	CZ	24	San Marino	SM	27
Denmark	DK	18	Slovakia	SK	24
Estonia	EE	20	Slovenia	SI	19
Finland	FI	18	Spain	ES	24
France	FR	27	Sweden	SE	24
Germany	DE	22	Switzerland	CH	21
Greece	GR	27	United Kingdom	GB	22
Hungary	HU	28	Vatican City	VA	22
Iceland	IS	26	Andorra	AD	24
Ireland	IE	22	Liechtenstein	LI	21
Italy	IT	27	Monaco	MC	27
Latvia	LV	21	Luxembourg	LU	20
Lithuania	LT	20	Malta	MT	31

Examples

```
// Valid IBANs
IBANValidator.IsValid("IT60X0542811101000000123456"); // Italy
IBANValidator.IsValid("DE89370400440532013000"); // Germany
IBANValidator.IsValid("GB29NWBK60161331926819"); // United Kingdom
```

```
IBANValidator.IsValid("FR7630006000011234567890189"); // France
IBANValidator.IsValid("ES9121000418450200051332"); // Spain
IBANValidator.IsValid("CH5604835012345678009"); // Switzerland
IBANValidator.IsValid("NO9386011117947"); // Norway

// Invalid IBANs
IBANValidator.IsValid(""); // false (empty)
IBANValidator.IsValid("IT00X0542811101000000123456"); // false (bad mod-97 checksum)
IBANValidator.IsValid("XX60X0542811101000000123456"); // false (unknown country XX)
IBANValidator.IsValid("US60X0542811101000000123456"); // false (non-SEPA country)
IBANValidator.IsValid("IT60X054281110100000012345"); // false (wrong length: 26 instead
of 27)
```


VAT Validation (Partita IVA)

Overview

`ItalianVatCodeValidator.IsValid(string vat, bool isConsumer, bool isFiscal)` validates Italian VAT codes (*Partita IVA*) according to the rules defined by the Agenzia delle Entrate.

```
using CodiceFiscale;
```

```
bool valid = ItalianVatCodeValidator.IsValid("00484960588", isConsumer: false,
isFiscal: false);
// true
```

The method is **static** and **thread-safe**. It never throws.

Parameters

Parameter	Type	Description
<code>vat</code>	<code>string</code>	The code to validate. Either an 11-digit Partita IVA or a 16-character Codice Fiscale (if <code>isFiscal</code> is <code>true</code>).
<code>isConsumer</code>	<code>bool</code>	<code>true</code> if the code belongs to a natural person (consumer). The first digit must be 8 or 9.
<code>isFiscal</code>	<code>bool</code>	<code>true</code> to also accept a 16-character Codice Fiscale as a valid identifier.

Validation algorithm

A standard 11-digit Partita IVA is validated as follows:

1. **Length** must be exactly 11 digits (all numeric)
2. **Consumer flag** — if `isConsumer: true`, first digit must be 8 or 9; if `false`, first digit must be 0–7
3. **Luhn-like checksum** (Agenzia delle Entrate algorithm):
 - Sum digits at **odd positions** (1, 3, 5, 7, 9) directly
 - Double digits at **even positions** (2, 4, 6, 8, 10); if the result ≥ 10 , subtract 9
 - The 11th digit must equal $(10 - (\text{sum} \% 10)) \% 10$

Mode combinations

isConsumer	isFiscal	Accepts
false	false	11-digit VAT, first digit 0–7
true	false	11-digit VAT, first digit 8–9
false	true	11-digit VAT (any) or valid 16-char Codice Fiscale
true	true	Consumer VAT (first digit 8–9) or valid Codice Fiscale

Examples

```
// Standard company VAT
```

```
ItalianVatCodeValidator.IsValid("00484960588", false, false); // true
```

```
ItalianVatCodeValidator.IsValid("10433218194", false, false); // true
```

```
// Consumer VAT (first digit 8 or 9)
```

```
ItalianVatCodeValidator.IsValid("85423511618", true, false); // true
```

```
ItalianVatCodeValidator.IsValid("91849593107", true, false); // true
```

```
// Codice Fiscale accepted as fiscal identifier
```

```
ItalianVatCodeValidator.IsValid("RSSMRA85T10A562S", false, true); // true
```

```
ItalianVatCodeValidator.IsValid("00484960588", false, true); // true (VAT  
also accepted)
```

```
// Invalid cases
```

```
ItalianVatCodeValidator.IsValid("", false, false); // false (empty)
```

```
ItalianVatCodeValidator.IsValid(" ", false, false); // false (whitespace)
```

```
ItalianVatCodeValidator.IsValid("0218208039", false, false); // false (10 digits,  
too short)
```

```
ItalianVatCodeValidator.IsValid("021820803900", false, false); // false (12 digits, too long)
```

```
ItalianVatCodeValidator.IsValid("02182080391", false, false); // false (bad checksum)
```

```
ItalianVatCodeValidator.IsValid("80025291002", false, false); // false (first digit 8,  
but isConsumer=false)
```

```
ItalianVatCodeValidator.IsValid("02182080390", true, false); // false (first digit 0,  
but isConsumer=true)
```

```
ItalianVatCodeValidator.IsValid("RSSMRA85T10A562S", false, false); // false (CF not accepted  
without isFiscal)
```

Changelog

All notable changes to this project will be documented in this file.

The format is based on [Keep a Changelog](#), and this project adheres to [Semantic Versioning](#).

[1.0.0](#) - 2026-06-01

Added

Codice Fiscale

- `CodiceFiscaleValidator.IsValid(string cf)` — full 5-step validation pipeline: regex format, birth date, Belfiore code, check digit
- `CodiceFiscaleParser.Parse(string cf)` — decodes a CF into `CodiceFiscaleData` (gender, date of birth, Belfiore code)
- `CodiceFiscaleParser.TryParse(string cf, out CodiceFiscaleData? data)` — non-throwing alternative to `Parse`
- `InvalidCodiceFiscaleException` — dedicated exception type for invalid CF inputs
- **Foreign-born support** — Belfiore codes starting with **Z** (e.g. **Z614** = Venezuela) are validated against a dataset of 261 foreign countries (active + historical/soppresso)
- `CodiceFiscaleData` — immutable `record` with `Gender`, `DateOfBirth`, `BelfioreCode`
- `Gender` enum with `Male` and `Female` values

IBAN

- `IBANValidator.IsValid(string iban)` — validates IBAN using the ISO 13616 mod-97 algorithm
- Supports all **36 SEPA countries** (EU27 + UK, Switzerland, Norway, Iceland, Liechtenstein, Monaco, San Marino, Andorra, Vatican City)
- Each country's expected IBAN length is enforced separately
- Spaces in the input are accepted and stripped before validation

Partita IVA

- `ItalianVatCodeValidator.IsValid(string vat, bool isConsumer, bool isFiscal)` — validates Italian VAT codes using the Luhn-like Agenzia delle Entrate algorithm
- `isConsumer` flag enforces first-digit rule (8–9 for natural persons, 0–7 for companies)
- `isFiscal` flag enables accepting a 16-character Codice Fiscale as a valid identifier

Datasets (embedded resources)

- `Resources/municipalities.json` — ~7 800 Italian comuni with Belfiore code, province, region, coordinates
- `Resources/foreign_countries.json` — 261 foreign country codes (active + historical soppresso states: Czechoslovakia, East Germany, USSR, Yugoslavia, ...)

- [Resources/check_code_map.json](#) — odd/even value table for the Codice Fiscale check digit algorithm

Infrastructure

- `DataStore` — internal `Lazy<T>` singleton cache; all three datasets are deserialized once on first use and cached for the lifetime of the process (thread-safe, no locks required)
- `Config` — centralized embedded resource name constants
- `CodiceFiscaleTokenizer` — internal regex-based CF tokenizer used by Validator and Parser

Documentation

- Full [DocFX](#) documentation site published to GitHub Pages
- API reference generated from XML doc comments (zero CS1591 warnings)
- Conceptual docs: introduction, getting started, CF structure, validation, parsing, IBAN, VAT

Notes

- All public methods are **static** and **thread-safe**
- No HTTP calls, no external services — all validation runs locally in-process
- Targets **.NET 9.0**

[0.1.0](#) - 2026-05-22

Added

- Initial project structure
- Basic Codice Fiscale validation (Italian municipalities only)
- Basic IBAN and VAT stubs (not yet implemented)